

RAFT Consensus Algorithm

Distributed Systems (Team 73)

Monish Rishabh

November 28, 2024

Table of Contents

Introduction to RAFT

Node Management

RAFT Server Implementation

RPC System

Server and User Interface

Snapshot Verification

Conclusion

Work Distribution

Work Distribution: Common Parts

Rishabh (2023121001)

Monish (2022111003)

What is RAFT?

- ▶ RAFT is a consensus algorithm designed for managing a replicated log.
- ▶ Ensures that multiple nodes (servers) agree on the same sequence of state transitions.
- ▶ Widely used in distributed systems for fault tolerance and consistency.
- ▶ **Key Components:**
 - ▶ **Leader Election**
 - ▶ **Log Replication**
 - ▶ **Safety and Fault Tolerance**

RAFT System Diagram

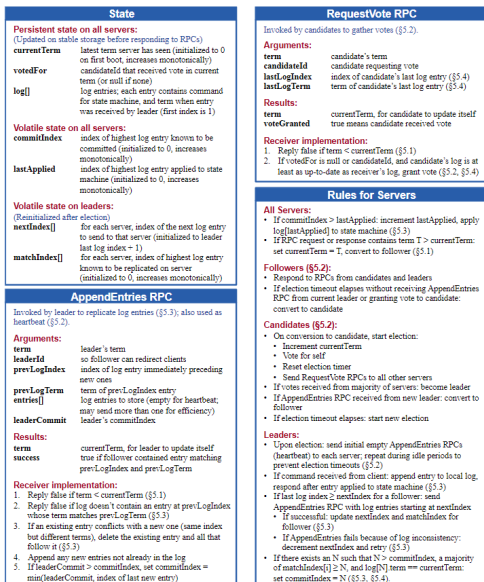


Figure: Diagram of RAFT Algorithm

Project Objectives

- ▶ Implement the RAFT consensus algorithm in Python.
- ▶ Ensure correct implementation of leader election, log replication, and safety guarantees.
- ▶ Develop mechanisms for log compaction and snapshotting.
- ▶ Create a robust RPC system for inter-node communication.
- ▶ Provide tools for verifying the correctness of snapshots.
- ▶ Demonstrate fault tolerance and data consistency through testing.

Node Class Overview

- ▶ **Singleton Pattern:** Ensures a single instance per node.
- ▶ **HTTP Server:** Handles incoming RPC requests.
- ▶ **Message Handling:** Processes messages based on endpoints.
- ▶ **Message Sending:** Facilitates communication between nodes with simulated message loss.
- ▶ **Endpoints Registration:** Dynamically registers handlers for different RPC endpoints.

Node Class Implementation

- ▶ **Singleton Pattern:** Ensures only one instance of a node exists.
- ▶ **HTTP Server Setup:**
 - ▶ Uses 'http.server.ThreadingHTTPServer' to handle concurrent requests.
 - ▶ Binds 'NodeHTTPRequestHandler' to manage incoming RPCs.
- ▶ **Message Sending:**
 - ▶ Sends messages via HTTP POST requests.
- ▶ **Simulates message loss based on 'message_{success}rate'.**
- ▶ **Endpoint Registration:**
 - ▶ Dynamically registers RPC handlers using endpoints.

Node Class code

- ▶ **Start Node:**
 - ▶ Initialize HTTP server on specified port.
 - ▶ Start server thread to handle incoming requests.
- ▶ **Register Endpoint:**
 - ▶ Map endpoint names to handler functions.
- ▶ **Handle Message:**
 - ▶ Receive message at endpoint.
 - ▶ Invoke corresponding handler.
- ▶ **Send Message:**
 - ▶ Send HTTP POST to target node's endpoint.
 - ▶ Handle response or simulate message loss.

RaftServer Class Overview

- ▶ Manages RAFT states: **Follower**, **Candidate**, **Leader**.
- ▶ Handles leader election, vote solicitation, and term management.
- ▶ Implements log replication and commit mechanisms.
- ▶ Manages snapshot creation and log compaction for efficiency.
- ▶ Ensures thread-safe operations using locks.
- ▶ Interacts with the KeyValueStore to apply commands.

State Management

- ▶ **Persistent State:** Maintains across restarts.
 - ▶ Current Term
 - ▶ Voted For
 - ▶ Log Entries
 - ▶ Snapshot Information
- ▶ **Volatile State:** Resets upon leader election.
 - ▶ Next Index
 - ▶ Match Index
 - ▶ Commit Index
 - ▶ Last Applied
- ▶ **Snapshotting:** Captures state to reduce log size and improve efficiency.

Leader Election Process

- ▶ **Trigger:** Follower times out without receiving heartbeats.
- ▶ **Actions:**
 1. Increment current term.
 2. Vote for self.
 3. Solicit votes from peers via 'RequestVote' RPC.
- ▶ **Outcome:**
 - ▶ Majority votes: Become Leader.
 - ▶ Otherwise: Revert to Follower.

Leader Election code

```
1 def start_election(self):
2     acquire_lock()
3     reset_election_timeout()
4     increment_term()
5     vote_for_self()
6     votes_granted = 1
7     release_lock()
8
9     for peer in peers:
10         term, vote = RequestVote(peer, current_term, self_id
11         , last_log_index, last_log_term)
12         if term > current_term:
13             become_follower(term)
14             return
15         if vote:
16             votes_granted += 1
17             if votes_granted > majority:
18                 break
19
20     acquire_lock()
21     if votes_granted > majority:
22         become_leader()
23     else:
24         become_follower(current_term)
25     release_lock()
```

Log Replication

- ▶ **Leader Actions:**

- ▶ Append commands to its own log.
- ▶ Send 'AppendEntries' RPCs to followers with new log entries.

- ▶ **Follower Actions:**

- ▶ Append received entries to their logs after validation.
- ▶ Reply to leader with success or failure.

- ▶ **Commit Mechanism:**

- ▶ Leader commits entries once a majority have replicated them.
- ▶ Applies committed entries to the state machine.

Log Replication code

```
1 def send_heartbeats(self):
2     for peer in peers:
3         prev_log_index = next_index[peer] - 1
4         prev_log_term = log[prev_log_index].term
5         entries = log[next_index[peer]:]
6         success = AppendEntries(peer, current_term, self_id,
7                                 prev_log_index, prev_log_term, entries, commit_index)
8         if success:
9             match_index[peer] = len(entries)
10            next_index[peer] = len(log) + 1
11            update_commit_index()
12        else:
13            next_index[peer] -= 1
14
15 def apply_committed_entries(self):
16     while last_applied < commit_index:
17         entry = log[last_applied]
18         apply_to_state_machine(entry.command)
19         last_applied += 1
```

Listing: Log Replication Process

Snapshotting and Log Compaction

- ▶ **Purpose:** Reduce log size by capturing the current state.
- ▶ **Trigger:** When log size exceeds a predefined threshold.
- ▶ **Process:**
 1. Create a snapshot of the current state machine.
 2. Truncate the log up to the snapshot index.
 3. Save the snapshot to persistent storage.
- ▶ **Benefits:**
 - ▶ Improves efficiency by reducing the amount of log data.
 - ▶ Facilitates quicker recovery and state synchronization for new nodes.

Snapshotting code

```
1 def create_snapshot(self):
2     acquire_lock()
3     snapshot_data = state_machine.snapshot()
4     snapshot = {
5         'data': snapshot_data,
6         'last_included_index': len(log) - 1,
7         'last_included_term': log[-1].term
8     }
9     save_snapshot(snapshot)
10    log = log[:snapshot['last_included_index'] + 1]
11    save_log()
12    release_lock()
```

Listing: Snapshot Creation Process

RPC Handling with Decorators

- ▶ Utilizes decorators to define and bind RPC methods seamlessly.
- ▶ Automatically handles serialization and deserialization of messages.
- ▶ Supports both class methods and standalone functions.
- ▶ Ensures type-safe communication between nodes.
- ▶ Simplifies the process of adding new RPC endpoints.

RPC Decorator Implementation

- ▶ **Decorator Function:** `rpc_call` wraps RPC methods.
- ▶ **Serialization:** Converts function arguments to a byte format for transmission.
- ▶ **Deserialization:** Converts received bytes back to function arguments.
- ▶ **Endpoint Binding:** Automatically registers the function as an endpoint handler.

```
1 def rpc_call(is_class_method=False):
2     def decorator(func):
3         endpoint = func.__name__.replace("_", "-")
4         def wrapper(*args, **kwargs):
5             if is_class_method: obj, port, *args = args
6             else: port, *args = args
7             if port is None:
8                 return func(*args, **kwargs)
9             serialized_msg = serialize(args, kwargs)
10            response = send_message(port, endpoint,
11                                   serialized_msg)
12            return deserialize(response)
13            register_endpoint(endpoint, func)
14            return wrapper
15    return decorator
```

Server Initialization

- ▶ **Command-Line Arguments:**

- ▶ Node ID
- ▶ Port to listen on
- ▶ List of peer ports
- ▶ Save file location
- ▶ Load from save file option

- ▶ **Logging Configuration:**

- ▶ Sets up logging with node-specific identifiers.

- ▶ **RAFT Server Initialization:**

- ▶ Creates an instance of 'RaftServer' with specified parameters.

- ▶ **Start Node:**

- ▶ Starts the HTTP server to listen for incoming RPCs.

Server Main Function code

```
1 if __name__ == "__main__":
2     parse_arguments()
3     configure_logging()
4     node_instance = Node(node_id, port, peers)
5     raft_server = RaftServer(node_id, peers, save_file,
6                               load_from_file)
7     node_instance.raft_server = raft_server
8     node_instance.start()
9
10    while running:
11        command = get_user_input()
12        handle_command(command)
```

Listing: Server Initialization code

Interactive Command-Line Interface

► Supported Commands:

- `send <target_port> to <endpoint> <message>`
- `set <key> <value>`
- `get <key>`
- `make-leader`
- `save <filename>`
- `stop`

► Functionality:

- Allows users to interact with the RAFT node in real-time.
- Enables sending custom messages, setting and getting key-value pairs, promoting to leader, saving state, and stopping the node.

Interactive CLI code

```
1 while running:
2     command = input("Node> ").strip()
3     if command.startswith("send"):
4         parse_and_send_message(command)
5     elif command.startswith("set"):
6         key, value = extract_key_value(command)
7         raft_server.client_set(key, value)
8     elif command.startswith("get"):
9         key = extract_key(command)
10        value = raft_server.client_get(key)
11        display_value(key, value)
12    elif command == "make-leader":
13        raft_server.force_leader()
14    elif command.startswith("save"):
15        filename = extract_filename(command)
16        raft_server.save_state(filename)
17    elif command == "stop":
18        node_instance.stop()
19        break
20    else:
21        display_unknown_command()
```

Listing: Command Handling code

Snapshot Verification Tool

- ▶ **Purpose:** Validate the integrity and correctness of snapshot files.
- ▶ **Functionality:**
 - ▶ Checks if the save file exists and is properly formatted.
 - ▶ Verifies that the log has been truncated correctly.
 - ▶ Ensures that snapshot data matches the key-value store.
- ▶ **Usage:**
 - ▶ Run the verification script with the path to the save file.
 - ▶ Receives detailed output on the verification status.

Conclusion

- ▶ Successfully implemented the RAFT consensus algorithm with full support for:
 - ▶ **Leader Election**
 - ▶ **Log Replication**
 - ▶ **Snapshotting and Log Compaction**
- ▶ Developed a robust RPC system facilitating reliable inter-node communication.
- ▶ Ensured data consistency and fault tolerance through comprehensive state management.
- ▶ Created verification tools to maintain and validate system integrity.
- ▶ Demonstrated the system's capability to handle failures and maintain consensus.

Work Distribution: Common Parts

Core Raft Consensus Logic:

- ▶ Developed the `RaftServer` class in `raft_server.py`, managing the overall Raft consensus algorithm including leader election, state transitions, and maintaining server state.
- ▶ Implemented state transitions through methods like `become_leader` and `become_follower`.

Rishabh's Contributions

Key-Value Store Implementation:

- ▶ Developed the `KeyValueStore` class in `raft_server.py`, including the `get` and `set` methods to handle client requests for data retrieval and modification.
- ▶ Ensured thread-safety within the key-value store operations using appropriate locking mechanisms.

Log Replication:

- ▶ Implemented log replication logic within the `RaftServer` class, particularly in the `AppendEntries` method to ensure that log entries are consistently replicated across all follower nodes.
- ▶ Managed the `match_index` and `next_index` for each peer to track replication progress.

Rishabh's Contributions (Continued)

Snapshots and Log Compaction:

- ▶ Designed and implemented the Snapshot class to represent snapshots of the state machine.
- ▶ Developed methods such as `create_snapshot` and `apply_snapshot` in `RaftServer` to handle log compaction and apply snapshots to the key-value store.
- ▶ Created the `verify_snapshot.py` script to validate the integrity and correctness of snapshots.

Monish's Contributions

Node Communication and RPC Handling:

- ▶ Designed and implemented the `Node` class in `node.py` to manage node instances, handle incoming RPCs, and facilitate communication between nodes.
- ▶ Developed the `rpc.py` module, including the `rpc_call` decorator to streamline the registration and handling of RPC methods.
- ▶ Implemented RPC handlers such as `RequestVote` and `AppendEntries` to process incoming consensus messages.

Monish's Contributions (Continued)

Server Initialization and Command-Line Interface:

- ▶ Developed the `server.py` script to serve as the entry point for starting Raft nodes, handling argument parsing, logging setup, and initializing server components.
- ▶ Implemented the command-line interface within `server.py` to allow user interactions such as sending messages, setting and getting key-value pairs, and manually promoting a node to leader.

Election and Heartbeat Mechanisms:

- ▶ Implemented election timers and heartbeat intervals within the `RaftServer` class to manage leader elections and maintain leadership through regular heartbeats.
- ▶ Managed election timeouts and handled the logic to reset or trigger elections based on node states and message receptions.

Monish's Contributions (Continued)

Log and State Management:

- ▶ Managed the persistent and volatile state of the server through the `ServerState`, `PersistentServerState`, and `VolatileLeaderState` data classes.
- ▶ Implemented methods to save and load server state to and from durable storage, ensuring persistence across restarts.

Thank You!

Questions?

Raft vs Paxos

- ▶ Largely, Raft is similar to Multi-Paxos with some key differences
 - ▶ No out of order updates (all updates follow the leader)
 - ▶ No retrying elections
- ▶ With minor changes, we can get Raft*, a refinement of Raft that can be implemented using Paxos building blocks.

Paxos vs Raft: Overview Comparison

Feature	Paxos	Raft
Design Goal	Consensus with correctness focus.	Understandable and practical consensus.
Leader Election	Implicit leader, complex election.	Explicit leader, straightforward process.
Understandability	Complex and abstract.	Easy to understand, clear terms.

Paxos vs Raft: Detailed Comparison

Feature	Paxos	Raft
Fault Tolerance	Majority operational.	Majority operational, clear failover.
Log Management	Multiple phases, abstract.	Leader manages replication, linear log.
Implementation Complexity	Higher complexity.	Lower complexity, structured approach.
Performance	Slower due to phases.	Faster with efficient replication.
Use Cases	Academic, foundational systems.	Industry adoption (etcd, Consul).